ANALYSE DU PROBLÈME

1. Contexte

Une agence de voyage à Cotonou veut informatiser la gestion de ses vols et de ses réservations de billets d'avion pour travailler plus vite et sans erreur.

2. Travail demandé

Vous êtes chargé de concevoir et de développer une application console permettant de gérer :

- La gestion des passagers
- La gestion des vols
- La gestion des réservations
- La gestion des paiements et des annulations

3. Analyse des besoins fonctionnels

L'application doit permettre à l'agent de l'agence de :

- Enregistrer un nouveau passager
- Enregistrer un nouveau vol
- Rechercher et afficher un passager ou un vol
- Créer une réservation pour un passager sur un vol
- Enregistrer le paiement d'une réservation
- Annuler une réservation
- Vérifier le nombre de places disponibles dans un vol

4. Acteurs du système

Acteur principal : L'agent de l'agence

C'est lui qui utilise l'application. Il enregistre les passagers, crée les vols, fait les réservations, enregistre les paiements et annule les réservations.

Acteur secondaire : Le passager

C'est le client de l'agence. Il ne manipule pas l'application. Il fournit ses informations, demande une réservation, paye ou demande une annulation.

5. Identification des classes (les objets à gérer)

À partir de l'énoncé, on identifie quatre objets principaux qui deviendront les classes dans la modélisation UML :

- **Passager** : la personne qui voyage
- **Vol** : le trajet proposé par l'agence
- **Réservation** : le lien entre un passager et un vol
- **Paiement** : le paiement qui valide une réservation

6. Règles de gestion à respecter

Ce sont les consignes imposées par l'énoncé. L'application doit les respecter obligatoirement.

Règle 1 : Un passager doit être enregistré avant de pouvoir réserver un billet. Sans enregistrement, aucune réservation n'est possible.

Règle 2 : Un vol ne peut recevoir plus de réservations que sa capacité. Le nombre de réservations doit toujours être inférieur ou égal au nombre de places du vol.

Règle 3 : Un passager ne peut pas réserver deux fois le même vol. Une seule réservation est autorisée par passager et par vol.

Règle 4 : Une réservation doit être associée à un seul passager et à un seul vol. Une réservation ne peut pas contenir deux passagers ni deux vols.

Règle 5 : Une réservation annulée ne peut plus être utilisée pour effectuer un paiement. Une fois annulée, la réservation est fermée.

Règle 6 : Une place est considérée comme définitivement réservée uniquement après le paiement. Tant que la réservation n'est pas payée, son statut reste « En attente » et la place n'est pas définitivement acquise.

Règle 7 : Une réservation payée et annulée doit être traitée selon le statut de remboursement prévu par l'application. L'application doit donc préciser si le passager est remboursé ou non.

7. Cycle de vie d'une réservation

Pour bien comprendre le fonctionnement, on retient l'ordre obligatoire suivant :

1. Enregistrement du passager
2. Création de la réservation, statut = En attente
3. Paiement de la réservation, statut = Payée, place définitivement réservée
4. Annulation possible, statut = Annulée, place libérée et remboursement traité si la réservation était payée

Sans réservation, il n'y a pas de paiement. Sans paiement, il n'y a pas de place définitive. Sans annulation, la place ne redevient pas libre.

Conclusion de l'analyse

L'analyse montre que le système repose sur quatre classes liées entre elles. Le passager doit d'abord être enregistré, puis il réserve un vol dans la limite des places disponibles, puis il paye pour valider définitivement sa place. L'annulation permet de libérer la place et de traiter le remboursement.

Cette analyse permet maintenant de passer à l'étape suivante : la modélisation UML, avec les attributs, les méthodes, les associations et les multiplicités.